

Assignment 3

CS 446-01

JASON STYS

CONTENTS

1. Abstract	3
2. Introduction	3
3. Architectural design	3
3.1. Logical topology	3
3.2. CIDR choice and trade offs	3
3.3. Why a private database tier is superior	4
4. Implementation procedure	4
4.1. Part 1 network foundation	4
4.2. Part 2 deploy compute instances	5
4.3. Part 3 advanced firewall configuration	6
4.4. Part 4 implement storage solutions	7
4.5. Part 5 verification	9
4.6. Part 6 global expansion and automation	9
5. Results	10
5.1. Web tier availability, database isolation, and admin access	10
5.2. Scope and environment issues	10
5.3. Troubleshooting captures	10
5.4. Storage evidence	10
5.5. EU deployment and early connectivity test	10
6. Discussion and analysis	11
6.1. Security posture strengths	11
6.2. Primary remaining risk and mitigation	11

6.3. IAP compared with a bastion host	12
6.4. SSD versus standard persistent disk	12
6.5. Operational lessons	12
7. Conclusion	14
8. References	15
References	15
9. Appendix	17
9.1. Lab prompt excerpt	17
9.2. Verification commands	17
9.3. EU private connectivity check	18

1. ABSTRACT

Implementing a secure multi tier network for TerraSync Media using Google Cloud. The solution used one custom mode VPC with regional subnets. A public web subnet hosted a web server with an external address and a private database subnet hosted a database server with no external address. Firewall rules enforced least privilege by allowing HTTP to the web tier, allowing database traffic only from the web tier, and limiting administrative access through narrow source ranges and IAP tunneling. Object storage provided a bucket for static images with a lifecycle policy that transitions older objects to Nearline. Block storage added an SSD persistent disk attached to the database VM and mounted at a dedicated directory. A second region deployment in Europe used a startup script to automate Nginx installation and content publishing. The completed environment demonstrates segmentation, least privilege firewalling, and a clear pathway for multi region growth.

2. INTRODUCTION

TerraSync Media distributes public content and stores sensitive metadata in a database. The network must enable public access to the web tier while keeping the database tier isolated. The design goals were

- (1) Provide reliable HTTP service to internet users
- (2) Keep the database unreachable from the public internet
- (3) Limit administrative access to a small trusted boundary
- (4) Use durable storage for both objects and block data
- (5) Prepare for a second region to support resilience and growth

The primary security principle in this lab was default deny. Each pathway was added only when it was required and each rule was limited to the smallest scope that still met the objective.

3. ARCHITECTURAL DESIGN

3.1. Logical topology. Figure 1 summarizes the final topology. The VPC spans regions while each subnet remains regional. In the US region the web server runs in a public subnet and the database server runs in a private subnet. In Europe an additional public subnet hosts a second web server for recovery and regional delivery.

3.2. CIDR choice and trade offs. The US tiers used 10.10.1.0/24 and 10.10.2.0/24. The EU tier used 10.20.1.0/24. A /24 supports 254 usable host addresses, which is enough room for

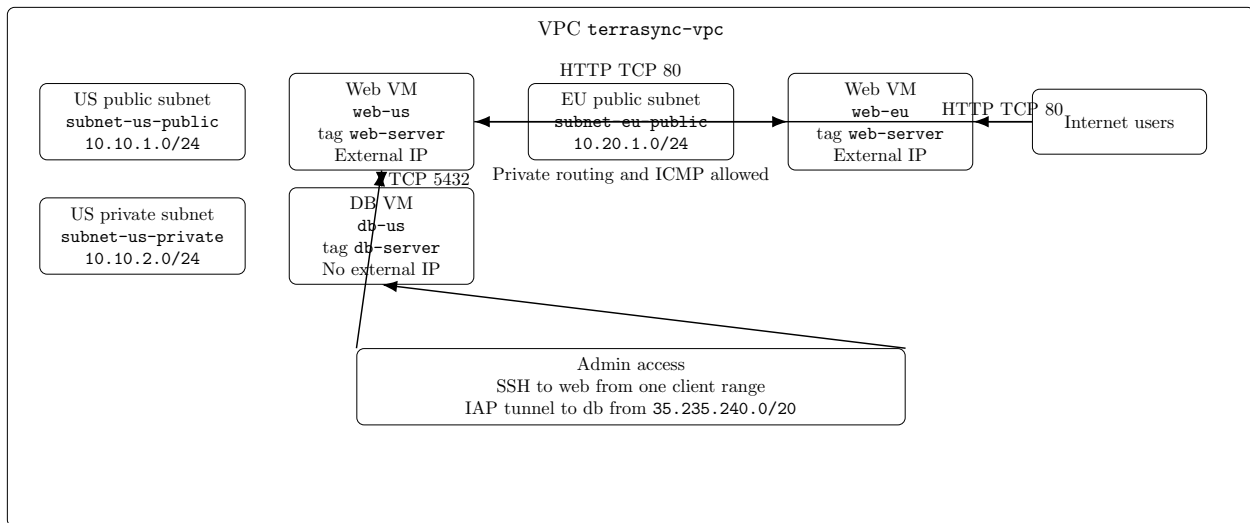


FIGURE 1. TerraSync architecture with global VPC, regional subnets, and least privilege flows.

a small tier and leaves space for additional instances, load balancers, and future services. Smaller ranges can force a redesign during growth. Much larger ranges waste address space and make it easier to write overly broad firewall rules.

3.3. Why a private database tier is superior. The database stores sensitive information and should expose as few interfaces as possible. A private subnet with no external address blocks direct internet access. It also limits scanning and brute force attempts because the host is reachable only through internal traffic paths that are explicitly allowed. This separation lets security controls concentrate on the web tier, which is the only public ingress point.

4. IMPLEMENTATION PROCEDURE

4.1. Part 1 network foundation. A custom mode VPC was created so subnets are explicitly defined. Two US subnets were created and the private subnet enabled private access to provider APIs.

LISTING 1. Create the VPC and US subnets

```
gcloud compute networks create terrasync-vpc \
  --subnet-mode=custom \
  --bgp-routing-mode=regional

gcloud compute networks subnets create subnet-us-public \
```

```
--network=terrasync-vpc \  
--range=10.10.1.0/24 \  
--region=us-central1
```

```
gcloud compute networks subnets create subnet-us-private \  
--network=terrasync-vpc \  
--range=10.10.2.0/24 \  
--region=us-central1 \  
--enable-private-ip-google-access
```

4.2. **Part 2 deploy compute instances.** The web server was placed in the public subnet and given an external address. The database server was placed in the private subnet and created with no external address. Tags separated the roles for firewall targeting.

LISTING 2. Create the web and database instances

```
ZONE=us-central1-a
```

```
gcloud compute instances create web-us \  
--zone=$ZONE \  
--machine-type=e2-micro \  
--network-interface=network=terrasync-vpc,subnet=subnet-us-public \  
--tags=web-server \  
--image-family=debian-12 \  
--image-project=debian-cloud  
  
gcloud compute instances create db-us \  
--zone=$ZONE \  
--machine-type=e2-micro \  
--network-interface=network=terrasync-vpc,subnet=subnet-us-private,no-address  
\  
--tags=db-server \  
--image-family=debian-12 \  
--image-project=debian-cloud
```

Nginx was installed on the web server and the default index page was replaced with a TerraSync banner.

LISTING 3. Install and configure Nginx on the web server

```
gcloud compute ssh web-us --zone=$ZONE

sudo apt-get update
sudo apt-get install -y nginx
echo "Welcome to TerraSync" | sudo tee /var/www/html/index.html
sudo systemctl enable --now nginx
exit
```

4.3. Part 3 advanced firewall configuration. Firewall rules were created to only allow the required traffic. The rules support five flows.

- (1) HTTP to the web tier on TCP port 80 from anywhere
- (2) Database access on TCP port 5432 only from instances tagged **web-server**
- (3) SSH to the web tier only from a single trusted client range
- (4) SSH to the database tier only from the IAP TCP forwarding range
- (5) ICMP within private ranges for controlled testing

LISTING 4. Firewall rules using least privilege

```
gcloud compute firewall-rules create terrasync-allow-http \
  --network=terrasync-vpc \
  --direction=INGRESS \
  --action=ALLOW \
  --rules=tcp:80 \
  --source-ranges=0.0.0.0/0 \
  --target-tags=web-server

gcloud compute firewall-rules create terrasync-allow-db-from-web \
  --network=terrasync-vpc \
  --direction=INGRESS \
  --action=ALLOW \
```

```
--rules=tcp:5432 \  
--source-tags=web-server \  
--target-tags=db-server
```

```
ADMIN_CIDR="YOUR_PUBLIC_IP/32"
```

```
gcloud compute firewall-rules create terrasync-allow-ssh-admin-web \  
  --network=terrasync-vpc \  
  --direction=INGRESS \  
  --action=ALLOW \  
  --rules=tcp:22 \  
  --source-ranges=$ADMIN_CIDR \  
  --target-tags=web-server
```

```
gcloud compute firewall-rules create terrasync-allow-ssh-iap-db \  
  --network=terrasync-vpc \  
  --direction=INGRESS \  
  --action=ALLOW \  
  --rules=tcp:22 \  
  --source-ranges=35.235.240.0/20 \  
  --target-tags=db-server
```

```
gcloud compute firewall-rules create terrasync-allow-icmp-internal-us \  
  --network=terrasync-vpc \  
  --direction=INGRESS \  
  --action=ALLOW \  
  --rules=icmp \  
  --source-ranges=10.10.0.0/16
```

4.4. **Part 4 implement storage solutions.** Object storage was used for static assets. A lifecycle policy moved older objects to Nearline after 90 days to reduce cost. Block storage used a 20 GB SSD persistent disk attached to the database VM. The disk was formatted and mounted so it can store database files.

LISTING 5. Create a bucket and set a lifecycle policy

```
BUCKET="terrasync-images-$DEVSHHELL_PROJECT_ID"
gsutil mb -l us-central1 gs://$BUCKET/

cat > lifecycle.json <<'JSON'
{
  "rule": [
    {
      "action": {"type": "SetStorageClass", "storageClass": "NEARLINE"},
      "condition": {"age": 90}
    }
  ]
}
JSON
gsutil lifecycle set lifecycle.json gs://$BUCKET/
```

LISTING 6. Create and attach a persistent disk

```
gcloud compute disks create db-us-data-disk \
  --size=20GB \
  --type=pd-ssd \
  --zone=$ZONE

gcloud compute instances attach-disk db-us \
  --disk=db-us-data-disk \
  --zone=$ZONE
```

LISTING 7. Format and mount the disk on the database VM

```
gcloud compute ssh db-us --zone=$ZONE --tunnel-through-iap

sudo lsblk
sudo mkfs.ext4 -F /dev/sdb
sudo mkdir -p /mnt/data/postgres
sudo mount /dev/sdb /mnt/data/postgres
```

```
sudo df -h | head
exit
```

4.5. **Part 5 verification.** Verification checked public service, isolation, admin access, and internal reachability. The key checks were

- (1) Verify the web page loads in a browser or via curl
- (2) Verify the database has no external address
- (3) Verify SSH access to the web server from the allowed range
- (4) Verify SSH access to the database through IAP tunneling
- (5) Verify ping and port tests using internal addresses
- (6) Verify the lifecycle policy and disk mount state

4.6. **Part 6 global expansion and automation.** A new EU subnet was added to the existing VPC. A new VM was created in Europe with a startup script that installs Nginx and publishes a region specific index page.

LISTING 8. Create the EU subnet and deploy an automated web server

```
gcloud compute networks subnets create subnet-eu-public \
  --network=terrasync-vpc \
  --region=europe-west1 \
  --range=10.20.1.0/24

EU_ZONE=europe-west1-b
cat > eu_startup.sh <<'SH'
#!/bin/bash
set -euxo pipefail
export DEBIAN_FRONTEND=noninteractive
apt-get update
apt-get install -y nginx
echo "Hello from TerraSync Europe!" > /var/www/html/index.html
systemctl enable --now nginx
SH
```

```
gcloud compute instances create web-eu \  
  --zone=$EU_ZONE \  
  --machine-type=e2-micro \  
  --network-interface=network=terrasync-vpc,subnet=subnet-eu-public \  
  --tags=web-server \  
  --metadata-from-file startup-script=eu_startup.sh \  
  --image-family=debian-12 \  
  --image-project=debian-cloud
```

5. RESULTS

This section presents the primary evidence captured during the lab.

5.1. Web tier availability, database isolation, and admin access. Figure 2 shows a successful HTTP response from the web server. It also shows that **db-us** has no NAT address while **web-us** has a NAT address. The same capture includes a direct SSH login to **web-us** and an IAP tunneled SSH login to **db-us**.

5.2. Scope and environment issues. Figure 3 shows an authentication scope failure when running **gcloud** from inside a VM. Resource management was completed in Cloud Shell where credentials and permissions matched the project settings.

5.3. Troubleshooting captures. Figure 4 shows that placeholder tokens were accidentally used in a ping command. Figure 5 shows a package update attempt that failed due to unreachable network paths. Figure 6 shows an initial attempt to validate port 5432 with netcat where the tool was not present and the database internal address was not substituted.

5.4. Storage evidence. Figure 7 shows bucket discovery and lifecycle output. The same capture also shows a disk mount check location. A final verification capture should show `/mnt/data/postgres` in the `df -h` output after the disk is formatted and mounted.

5.5. EU deployment and early connectivity test. Figure 8 shows EU zone selection, startup script creation, EU instance creation, and an ICMP rule for internal testing. Figure 9 shows an early ping test from **web-us** to the EU internal address.

```

jasonstys27@cloudshell:~ (assignment-3-486322)$ gcloud compute instances describe db-us --zone=... --format="get(zone)"
ERROR: (gcloud.compute.instances.describe) Could not fetch resource:
~ Invalid value for field 'zone': '...'. Must be a match of regex '[a-z]([0-9]{0,61}[a-z0-9])?'

jasonstys27@cloudshell:~ (assignment-3-486322)$ ZONE=us-central1-a
WEB=web-us
DB=db-us

WEB_IP=$(gcloud compute instances describe WEB --zone=$ZONE --format='get(networkInterfaces[0].accessConfigs[0].natIP)')
DB_INT_IP=$(gcloud compute instances describe SDB --zone=$ZONE --format='get(networkInterfaces[0].networkIP)')

echo "WEB public IP: $WEB_IP"
echo "DB internal IP: $DB_INT_IP"
WEB public IP: 34.135.9.243
DB internal IP: 10.10.2.2
jasonstys27@cloudshell:~ (assignment-3-486322)$ curl -i "http://$WEB_IP" | head
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
             %    Upload    Download             Total       Spent    Left   Speed
100  21  100    21    0    0   153      0 --:--:-- --:--:-- --:--:--   153
HTTP/1.1 200 OK
Server: nginx/1.22.1
Date: Mon, 09 Feb 2026 06:13:36 GMT
Content-Type: text/html
Content-Length: 21
Last-Modified: Mon, 09 Feb 2026 05:14:36 GMT
Connection: keep-alive
ETag: "69866d2e-15"
Accept-Ranges: bytes

jasonstys27@cloudshell:~ (assignment-3-486322)$ gcloud compute instances list --filter="name=($WEB $DB)" --format="table(name,zone,networkInterfaces[0].networkIP,networkInterfaces[0].accessConfigs[0].natIP)"
NAME: db-us
NAME: web-us
ZONE: us-central1-a
ZONE: us-central1-a
NETWORK_IP: 10.10.1.2
NETWORK_IP: 10.10.1.2
NAT_IP: 34.135.9.243
NAT_IP: 34.135.9.243
jasonstys27@cloudshell:~ (assignment-3-486322)$ gcloud compute ssh WEB --zone=$ZONE
Linux web-us 6.1.0-42-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.159-1 (2025-12-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Feb 9 06:03:30 2026 from 34.169.175.96
jasonstys27@web-us:~$ exit
logout
Connection to 34.135.9.243 closed.
jasonstys27@cloudshell:~ (assignment-3-486322)$ gcloud compute ssh SDB --zone=$ZONE --tunnel-through-iap
WARNING:

To increase the performance of the tunnel, consider installing Numpy. For instructions,
please see https://cloud.google.com/iap/docs/using-tcp-forwarding#increasing_the_tcp_upload_bandwidth

Linux db-us 6.1.0-42-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.159-1 (2025-12-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Feb 9 06:08:31 2026 from 35.235.242.218
jasonstys27@sdb-us:~$ exit
logout

```

FIGURE 2. HTTP success from **web-us**, database has no NAT address, and IAP tunneled SSH to **db-us**.

6. DISCUSSION AND ANALYSIS

6.1. Security posture strengths. The architecture separates public and private workloads into different subnets. The database host has no external address, which removes direct public reachability. Firewall rules are scoped with tags so only intended instances receive each allowed port. Administrative access to the database uses IAP tunneling, which keeps SSH closed to the public internet and ties access to identity and project policy.

6.2. Primary remaining risk and mitigation. The remaining risk is compromise of the web server. The web tier is a public entry point and it can reach the database port by design. A production deployment should add controls that reduce the impact of a web tier compromise. Use strict patching, least privilege service accounts, restricted database credentials, and continuous monitoring. Use a managed database service where possible. Add application level authentication and rate limiting at the web layer. Place a load balancer in front of the web tier and use a

```
jasonstys27@web-us:~$ gcloud compute instances describe web-us \
--zone=us-central1-a \
--format="get(networkInterfaces[0].accessConfigs[0].natIP)"
ERROR: (gcloud.compute.instances.describe) Could not fetch resource:
- Request had insufficient authentication scopes.

jasonstys27@web-us:~$ gcloud compute instances create db-us \
--zone=us-central1-a \
--machine-type=e2-micro \
--network-interface=network=terrasync-vpc,subnet=subnet-us-private,no-address \
--tags=db-server \
--image-family=debian-12 \
--image-project=debian-cloud
ERROR: (gcloud.compute.instances.create) Could not fetch resource:
- Request had insufficient authentication scopes.

jasonstys27@web-us:~$ gcloud compute instances list --filter="name:(web-us db-us)"
WARNING: Some requests did not succeed.
- Request had insufficient authentication scopes.

WARNING: The following filter keys were not present in any resource : name
Listed 0 items.
jasonstys27@web-us:~$
```

FIGURE 3. Authentication scope failure when attempting to run `gcloud` from within a VM.

web application firewall rule set. Add network egress rules that reduce outbound access from the database subnet.

6.3. IAP compared with a bastion host. A bastion host adds an always running VM that requires patching and monitoring. It also creates a new public address. IAP tunneling reduces network exposure because the target host stays private. It also centralizes policy and supports audit logs.

6.4. SSD versus standard persistent disk. Database workloads often include many small random reads and writes. SSD persistent disks provide lower latency and higher IOPS than standard disks. Standard disks cost less and can still perform well for sequential workloads. This lab selected SSD to better match database access patterns.

6.5. Operational lessons. Several issues came from environment assumptions. Running `gcloud` inside a VM can fail due to limited scopes. Placeholder values such as `10.10.2.x` must be replaced with actual internal addresses. Private instances can have limited package access when outbound internet is not available. In that case validation can still proceed using built in tools such as `/dev/tcp` or a simple Python listener.

FIGURE 4. Ping command run with a placeholder value instead of the real internal address.

FIGURE 5. Package update output showing unreachable network paths during installation.

FIGURE 6. Early netcat based port validation attempt where the tool or the address was not available.

This delivered a working baseline for a secure cloud deployment. The design created a custom mode VPC, separated the web and database tiers into different subnets, limited ingress with tag based firewall rules, and used IAP tunneling for private administration. Storage was implemented with a bucket and lifecycle policy for objects and a persistent disk intended for database data. The expansion step demonstrated multi region design and automated configuration through a startup script. The same approach can be extended with managed load balancing, autoscaling, and centralized logging to support production needs while keeping the security boundary clear.

```

jasonstys27@web-us:~$ nc -zv 10.10.2.X 5432
nc: getaddrinfo for host "10.10.2.X" port 5432: Name or service not known
jasonstys27@web-us:~$ exit
logout
Connection to 34.135.9.243 closed.
jasonstys27@cloudshell:~ (assignment-3-486322) $ BUCKET=$(gsutil ls | grep -ml '^gs://terrasync-images-')
echo "$BUCKET"
gs://terrasync-images-assignment-3-486322/
jasonstys27@cloudshell:~ (assignment-3-486322) $ gsutil lifecycle get "$BUCKET"
{"rule": [{"action": {"storageClass": "NEARLINE", "type": "SetStorageClass"}, "condition": {"age": 90}}]}
jasonstys27@cloudshell:~ (assignment-3-486322) $ gcloud compute ssh $DB --zone=$ZONE --tunnel-through-iap
WARNING:

To increase the performance of the tunnel, consider installing NumPy. For instructions,
please see https://cloud.google.com/iap/docs/using-tcp-forwarding#increasing_the_tcp_upload_bandwidth

Linux db-us 6.1.0-42-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.159-1 (2025-12-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Feb  9 06:15:05 2026 from 35.235.242.217
jasonstys27@db-us:~$ df -h | head -n 1
df -h | grep -E "/mnt/data/postgres|Filesystem"
Filesystem      Size  Used Avail Use% Mounted on
Filesystem      Size  Used Avail Use% Mounted on
jasonstys27@db-us:~$ exit
logout
Connection to compute.214319386539118423 closed.
jasonstys27@cloudshell:~ (assignment-3-486322) $ gcloud compute networks subnets create subnet-eu-public \
--network=terrasync-vpc \
--region=europe-west1 \
--range=10.20.1.0/24
Created [https://www.googleapis.com/compute/v1/projects/assignment-3-486322/regions/europe-west1/subnetworks/subnet-eu-public].
NAME: subnet-eu-public
REGION: europe-west1
NETWORK: terrasync-vpc
RANGE: 10.20.1.0/24
STACK_TYPE: IPV4_ONLY
IPV6_ACCESS_TYPE:
INTERNAL_IPV6_PREFIX:
EXTERNAL_IPV6_PREFIX:
jasonstys27@cloudshell:~ (assignment-3-486322) $ gcloud compute networks subnets describe subnet-eu-public \
--region=europe-west1 \
--format="table(name,region,ipCidrRange,network)"
NAME: subnet-eu-public
REGION: https://www.googleapis.com/compute/v1/projects/assignment-3-486322/regions/europe-west1
IP_CIDR_RANGE: 10.20.1.0/24
NETWORK: https://www.googleapis.com/compute/v1/projects/assignment-3-486322/global/networks/terrasync-vpc

```

FIGURE 7. Bucket lifecycle output and storage verification commands.

8. REFERENCES

REFERENCES

1. *Cloud Storage lifecycle management documentation*, Google Cloud (2026; <https://cloud.google.com/storage/docs/lifecycle>).
2. *Persistent Disk documentation*, Google Cloud (2026; <https://cloud.google.com/compute/docs/disks>).
3. *Using TCP forwarding with Identity Aware Proxy*, Google Cloud (2026; <https://cloud.google.com/iap/docs/using-tcp-forwarding>).
4. *Virtual Private Cloud documentation*, Google Cloud (2026; <https://cloud.google.com/vpc/docs/overview>).

```

jasonstys27@cloudshell:~ (assignment-3-486322)$ gcloud compute zones list --filter="region:(europa-west1)" --format="value(name)"
europa-west1-b
europa-west1-d
europa-west1-c
WARNING: --filter : operator evaluation is changing for consistency across Google APIs. region:europa-west1 currently matches but will not match in the near future. Run 'gcloud topic filters' for details.
europa-west10-a
europa-west10-b
europa-west10-c
europa-west12-a
europa-west12-b
europa-west12-c
jasonstys27@cloudshell:~ (assignment-3-486322)$ EU_ZONE=europa-west1-b
jasonstys27@cloudshell:~ (assignment-3-486322)$ cat > eu_startup.sh <<'SH'
#!/bin/bash
set -exo pipefail

export DEBIAN_FRONTEND=noninteractive
apt-get update
apt-get install -y nginx

echo "Hello from TerraSync Europe!" > /var/www/html/index.html
systemctl enable --now nginx
SH
jasonstys27@cloudshell:~ (assignment-3-486322)$ gcloud compute instances create web-eu \
--zone="$EU_ZONE" \
--machine-type=e2-micro \
--network-interface=network=terrasync-vpc,subnet=subnet-eu-public \
--tags=web-server \
--metadata-from-file startup-script=eu_startup.sh \
--image-family=debian-12 \
--image-project=debian-cloud
Created [https://www.googleapis.com/compute/v1/projects/assignment-3-486322/zones/europa-west1-b/instances/web-eu].
NAME: web-eu
ZONE: europa-west1-b
MACHINE_TYPE: e2-micro
PREEMPTIBLE:
INTERNAL_IP: 10.20.1.2
EXTERNAL_IP: 35.195.3.74
STATUS: RUNNING
jasonstys27@cloudshell:~ (assignment-3-486322)$ EU_WEB_IP=$(gcloud compute instances describe web-eu --zone="$EU_ZONE" --format="get(networkInterfaces[0].accessConfigs[0].natIP)")
echo "EU_WEB_IP"
curl -s "http://$EU_WEB_IP"
35.195.3.74
jasonstys27@cloudshell:~ (assignment-3-486322)$ gcloud compute firewall-rules create terrasync-allow-icmp-internal-eu \
--network=terrasync-vpc \
--direction=INGRESS \
--action=ALLOW \
--rules=icmp \
--source-ranges=10.20.1.0/24
Creating firewall...working...Created [https://www.googleapis.com/compute/v1/projects/assignment-3-486322/global/firewalls/terrasync-allow-icmp-internal-eu].
Creating firewall...done.
NAME: terrasync-allow-icmp-internal-eu
NETWORK: terrasync-vpc
DIRECTION: INGRESS
PRIORITY: 1000
ALLOW: icmp
DENY:
DISABLED: False
jasonstys27@cloudshell:~ (assignment-3-486322)$ EU_INT_IP=$(gcloud compute instances describe web-eu --zone="$EU_ZONE" --format="get(networkInterfaces[0].networkIP)")
echo "web-eu internal IP: $EU_INT_IP"
web-eu internal IP: 10.20.1.2

```

FIGURE 8. EU deployment using a startup script and ICMP rule creation.

```

jasonstys27@cloudshell:~ (assignment-3-486322)$ US_ZONE=us-central1-a
gcloud compute ssh web-us --zone="$US_ZONE"
Linux web-us 6.1.0-42-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.159-1 (2025-12-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Feb  9 06:16:49 2026 from 34.169.175.96
jasonstys27@web-us:~$ ping -c 6 <EU_INT_IP>
-bash: syntax error near unexpected token `newline'
jasonstys27@web-us:~$ ping -c 6 10.20.1.2
PING 10.20.1.2 (10.20.1.2) 56(84) bytes of data.
ex
--- 10.20.1.2 ping statistics ---
6 packets transmitted, 0 received, 100% packet loss, time 5119ms

jasonstys27@web-us:~$ exit
logout

```

FIGURE 9. Early ping test from web-us to the EU internal address.

5. *VPC firewall rules documentation*, Google Cloud (2026; <https://cloud.google.com/firewall/docs/firewalls>).

9. APPENDIX

1. Create Custom VPC

```
gcloud compute networks create terrasync-vpc \  
  --subnet-mode=custom \  
  --bgp-routing-mode=regional
```

2. Create US Subnets

Public Subnet (Web)

```
gcloud compute networks subnets create subnet-us-public \  
  --network=terrasync-vpc \  
  --range=10.10.1.0/24 \  
  --region=us-central1
```

Private Subnet (DB)

```
gcloud compute networks subnets create subnet-us-private \  
  --network=terrasync-vpc \  
  --range=10.10.2.0/24 \  
  --region=us-central1 \  
  --enable-private-ip-google-access
```

FIGURE 10. The initial VPC and subnet creation commands.

9.1. Lab prompt excerpt.

9.2. Verification commands.

WEB=web-us

```
DB=db-us
```

```
ZONE=us-central1-a
```

```
WEB_IP="$(gcloud compute instances describe $WEB --zone=$ZONE \
--format='get(networkInterfaces[0].accessConfigs[0].natIP)')"
```

```
DB_INT_IP="$(gcloud compute instances describe $DB --zone=$ZONE \
--format='get(networkInterfaces[0].networkIP)')"
```

```
curl -i "http://$WEB_IP" | head
```

```
gcloud compute ssh $WEB --zone=$ZONE
```

```
ping -c 4 $DB_INT_IP
```

```
# Port check without extra packages
```

```
timeout 2 bash -c "</dev/tcp/$DB_INT_IP/5432" && echo "Port open" || echo
    "Port closed"
```

```
exit
```

9.3. EU private connectivity check.

```
EU_ZONE=europe-west1-b
```

```
EU_INT_IP="$(gcloud compute instances describe web-eu --zone=$EU_ZONE \
--format='get(networkInterfaces[0].networkIP)')"
```

```
gcloud compute ssh web-us --zone=$ZONE
```

```
ping -c 6 $EU_INT_IP
```

```
exit
```
